

DESIGN PRINCIPLES AND PATTERNS

Exercise 1: Implementing the Singleton Pattern

Scenario:

You need to ensure that a logging utility class in your application has only one instance throughout the application lifecycle to ensure consistent logging.

Steps:

1. **Create a New Java Project:**
 - Create a new Java project named **SingletonPatternExample**.
2. **Define a Singleton Class:**
 - Create a class named **Logger** that has a private static instance of itself.
 - Ensure the constructor of **Logger** is private.
 - Provide a public static method to get the instance of the **Logger** class.
3. **Implement the Singleton Pattern:**
 - Write code to ensure that the **Logger** class follows the Singleton design pattern.
4. **Test the Singleton Implementation:**
 - Create a test class to verify that only one instance of **Logger** is created and used across the application.

PROGRAM:

SingletonTest.java

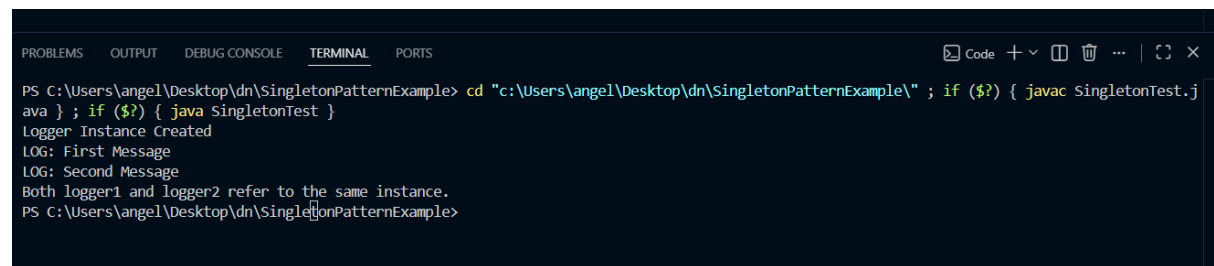
```
public class SingletonTest {
    public static void main(String[] args) {
        Logger logger1 = Logger.getInstance();
        logger1.log("First Message");
        Logger logger2 = Logger.getInstance();
        logger2.log("Second Message");
        if (logger1 == logger2) {
            System.out.println("Both logger1 and logger2 refer to the same instance.");
        } else {
            System.out.println("Different instances created.");
        }
    }
}
```

Logger.java

```
public class Logger {
    private static Logger instance;
    private Logger() {
        System.out.println("Logger Instance Created");
    }
}
```

```
}  
public static Logger getInstance() {  
    if (instance == null) {  
        instance = new Logger();  
    }  
    return instance;  
}  
public void log(String message) {  
    System.out.println("LOG: " + message);  
}  
}
```

OUTPUT:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
PS C:\Users\angel\Desktop\dn\SingletonPatternExample> cd "c:\Users\angel\Desktop\dn\SingletonPatternExample\" ; if ($?) { javac SingletonTest.java } ; if ($?) { java SingletonTest }  
Logger Instance Created  
LOG: First Message  
LOG: Second Message  
Both logger1 and logger2 refer to the same instance.  
PS C:\Users\angel\Desktop\dn\SingletonPatternExample>
```

Exercise 2: Implementing the Factory Method Pattern

Scenario:

You are developing a document management system that needs to create different types of documents (e.g., Word, PDF, Excel). Use the Factory Method Pattern to achieve this.

Steps:

1. **Create a New Java Project:**
 - Create a new Java project named **FactoryMethodPatternExample**.
2. **Define Document Classes:**
 - Create interfaces or abstract classes for different document types such as **WordDocument**, **PdfDocument**, and **ExcelDocument**.
3. **Create Concrete Document Classes:**
 - Implement concrete classes for each document type that implements or extends the above interfaces or abstract classes.
4. **Implement the Factory Method:**
 - Create an abstract class **DocumentFactory** with a method **createDocument()**.
 - Create concrete factory classes for each document type that extends **DocumentFactory** and implements the **createDocument()** method.
5. **Test the Factory Method Implementation:**
 - Create a test class to demonstrate the creation of different document types using the factory method.

PROGRAM:

Document.java

```
public interface Document {  
    void open();  
}
```

DocumentFactory.java

```
public abstract class DocumentFactory {  
    public abstract Document createDocument();  
}
```

ExcelDocument.java

```
public class ExcelDocument implements Document {  
    @Override  
    public void open() {  
        System.out.println("Excel Document Opened");  
    }  
}
```

ExcelFactory.java

```
public class ExcelFactory extends DocumentFactory {
    @Override
    public Document createDocument() {
        return new ExcelDocument();
    }
}
```

FactoryTest.java

```
public class FactoryTest {
    public static void main(String[] args) {
        DocumentFactory wordFactory = new WordFactory();
        Document word = wordFactory.createDocument();
        word.open();
        DocumentFactory pdfFactory = new PdfFactory();
        Document pdf = pdfFactory.createDocument();
        pdf.open();
        DocumentFactory excelFactory = new ExcelFactory();
        Document excel = excelFactory.createDocument();
        excel.open();
    }
}
```

PdfDocument.java

```
public class PdfDocument implements Document {
    @Override
    public void open() {
        System.out.println("PDF Document Opened");
    }
}
```

PdfFactory.java

```
public class PdfFactory extends DocumentFactory {
    @Override
    public Document createDocument() {
        return new PdfDocument();
    }
}
```

WordDocument.java

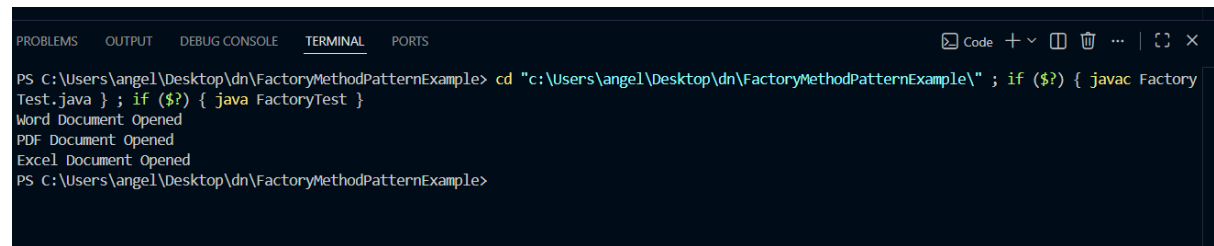
```
public class WordDocument implements Document {
    @Override
    public void open() {
        System.out.println("Word Document Opened");
    }
}
```

WordFactory.java

```
public class WordFactory extends DocumentFactory {
    @Override
    public Document createDocument() {
        return new WordDocument();
    }
}
```

```
}  
}
```

OUTPUT:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Code + - [ ] [ ] ... | [ ] [ ] X  
PS C:\Users\angel\Desktop\dn\FactoryMethodPatternExample> cd "c:\Users\angel\Desktop\dn\FactoryMethodPatternExample\" ; if ($?) { javac Factory  
Test.java } ; if ($?) { java FactoryTest }  
Word Document Opened  
PDF Document Opened  
Excel Document Opened  
PS C:\Users\angel\Desktop\dn\FactoryMethodPatternExample>
```

DATA STRUCTURES AND ALGORITHMS

Exercise 2: E-commerce Platform Search Function

Scenario:

You are working on the search functionality of an e-commerce platform. The search needs to be optimized for fast performance.

Steps:

1. **Understand Asymptotic Notation:**
 - Explain Big O notation and how it helps in analyzing algorithms.
 - Describe the best, average, and worst-case scenarios for search operations.
2. **Setup:**
 - Create a class **Product** with attributes for searching, such as **productId**, **productName**, and **category**.
3. **Implementation:**
 - Implement linear search and binary search algorithms.
 - Store products in an array for linear search and a sorted array for binary search.
4. **Analysis:**
 - Compare the time complexity of linear and binary search algorithms.
 - Discuss which algorithm is more suitable for your platform and why.

PROGRAM:

BinarySearchDemo.java

```
public class BinarySearchDemo {
    public static Product binarySearch(Product[] products, int id) {
        int low = 0;
        int high = products.length - 1;
        while (low <= high) {
            int mid = (low + high) / 2;
            if (products[mid].productId == id)
                return products[mid];
            else if (id < products[mid].productId)
                high = mid - 1;
            else
                low = mid + 1;
        }
        return null;
    }

    public static void main(String[] args) {
        Product[] products = {
            new Product(101, "Laptop", "Electronics"),
            new Product(102, "Phone", "Electronics"),
        }
    }
}
```

```

        new Product(103, "Shoes", "Fashion"),
        new Product(104, "Watch", "Accessories")
    };
    Product result = binarySearch(products, 103);
    if (result != null)
        System.out.println("Found: " + result.productName);
    else
        System.out.println("Not Found");
    }
}

```

LinearSearchDemo.java

```

public class LinearSearchDemo {
    public static Product linearSearch(Product[] products, int id) {
        for (Product p : products) {
            if (p.productId == id) {
                return p;
            }
        }
        return null;
    }
    public static void main(String[] args) {
        Product[] products = {
            new Product(101, "Laptop", "Electronics"),
            new Product(102, "Phone", "Electronics"),
            new Product(103, "Shoes", "Fashion")
        };
        Product result = linearSearch(products, 102);
        if (result != null)
            System.out.println("Found: " + result.productName);
        else
            System.out.println("Not Found");
    }
}

```

Product.java

```

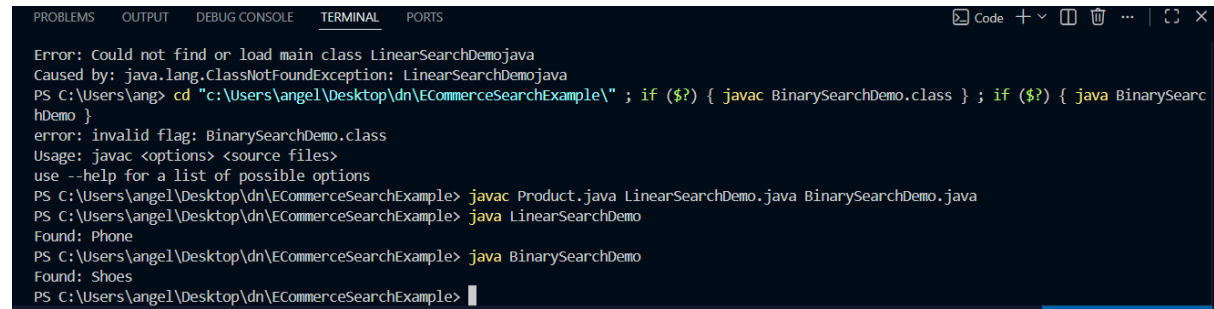
public class Product {

    int productId;
    String productName;
    String category;

    public Product(int productId, String productName, String category) {
        this.productId = productId;
        this.productName = productName;
        this.category = category;
    }
}

```

OUTPUT:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Error: Could not find or load main class LinearSearchDemojava
Caused by: java.lang.ClassNotFoundException: LinearSearchDemojava
PS C:\Users\angel> cd "c:\Users\angel\Desktop\dn\ECommerceSearchExample\" ; if ($?) { javac BinarySearchDemo.class } ; if ($?) { java BinarySearchDemo }
error: invalid flag: BinarySearchDemo.class
Usage: javac <options> <source files>
use --help for a list of possible options
PS C:\Users\angel\Desktop\dn\ECommerceSearchExample> javac Product.java LinearSearchDemo.java BinarySearchDemo.java
PS C:\Users\angel\Desktop\dn\ECommerceSearchExample> java LinearSearchDemo
Found: Phone
PS C:\Users\angel\Desktop\dn\ECommerceSearchExample> java BinarySearchDemo
Found: Shoes
PS C:\Users\angel\Desktop\dn\ECommerceSearchExample> 
```


Exercise 7: Financial Forecasting

Scenario:

You are developing a financial forecasting tool that predicts future values based on past data.

Steps:

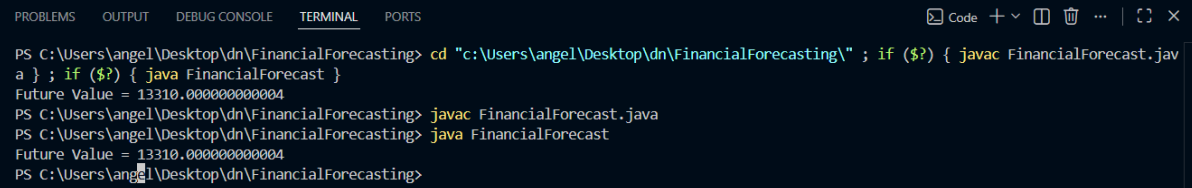
1. **Understand Recursive Algorithms:**
 - Explain the concept of recursion and how it can simplify certain problems.
2. **Setup:**
 - Create a method to calculate the future value using a recursive approach.
3. **Implementation:**
 - Implement a recursive algorithm to predict future values based on past growth rates.
4. **Analysis:**
 - Discuss the time complexity of your recursive algorithm.
 - Explain how to optimize the recursive solution to avoid excessive computation.

PROGRAM:

FinacialForecast.java

```
public class FinancialForecast {
    public static double forecast(double currentValue,
                                  double growthRate,
                                  int years) {
        if (years == 0) {
            return currentValue;
        }
        return forecast(currentValue * (1 + growthRate),
                        growthRate,
                        years - 1);
    }
    public static void main(String[] args) {
        double currentValue = 10000;
        double growthRate = 0.10; // 10%
        int years = 3;
        double futureValue =
            forecast(currentValue, growthRate, years);
        System.out.println("Future Value = " + futureValue);
    }
}
```

OUTPUT:



The screenshot shows a terminal window with the following content:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\angel\Desktop\dn\FinancialForecasting> cd "c:\Users\angel\Desktop\dn\FinancialForecasting\" ; if ($?) { javac FinancialForecast.java } ; if ($?) { java FinancialForecast }
Future Value = 13310.000000000004
PS C:\Users\angel\Desktop\dn\FinancialForecasting> javac FinancialForecast.java
PS C:\Users\angel\Desktop\dn\FinancialForecasting> java FinancialForecast
Future Value = 13310.000000000004
PS C:\Users\angel\Desktop\dn\FinancialForecasting>
```

PL/SQL PROGRAMMING

Exercise 1: Control Structures

Scenario 1: The bank wants to apply a discount to loan interest rates for customers above 60 years old.

- **Question:** Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates.

Scenario 2: A customer can be promoted to VIP status based on their balance.

- **Question:** Write a PL/SQL block that iterates through all customers and sets a flag IsVIP to TRUE for those with a balance over \$10,000.

Scenario 3: The bank wants to send reminders to customers whose loans are due within the next 30 days.

- **Question:** Write a PL/SQL block that fetches all loans due in the next 30 days and prints a reminder message for each customer.

Scenario 1: Apply 1% Interest Rate Discount for Customers Above 60

```
BEGIN
  FOR cust IN (
    SELECT CustomerID
    FROM Customers
    WHERE Age > 60
  )
  LOOP
    UPDATE Loans
    SET InterestRate = InterestRate - 1
    WHERE CustomerID = cust.CustomerID;
  END LOOP;

  COMMIT;
END;
/
```

Scenario 2: Promote Customers to VIP Status

```
BEGIN
  FOR cust IN (
    SELECT CustomerID
    FROM Customers
    WHERE Balance > 10000
  )
  LOOP
    UPDATE Customers
    SET IsVIP = 'TRUE'
    WHERE CustomerID = cust.CustomerID;
  END LOOP;

  COMMIT;
END;
```

/

Scenario 3: Send Loan Due Reminders

BEGIN

FOR loan_rec IN (

SELECT c.Name,

l.LoanID,

l.DueDate

FROM Customers c

JOIN Loans l

ON c.CustomerID = l.CustomerID

WHERE l.DueDate BETWEEN SYSDATE AND SYSDATE + 30

)

LOOP

DBMS_OUTPUT.PUT_LINE(

'Reminder: Dear ' || loan_rec.Name ||

', your Loan ID ' || loan_rec.LoanID ||

' is due on ' || TO_CHAR(loan_rec.DueDate,'DD-MON-YYYY')

);

END LOOP;

END;

/

Exercise 3: Stored Procedures

Scenario 1: The bank needs to process monthly interest for all savings accounts.

- **Question:** Write a stored procedure **ProcessMonthlyInterest** that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

Scenario 2: The bank wants to implement a bonus scheme for employees based on their performance.

- **Question:** Write a stored procedure **UpdateEmployeeBonus** that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

Scenario 3: Customers should be able to transfer funds between their accounts.

- **Question:** Write a stored procedure **TransferFunds** that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.

Scenario 1: Process Monthly Interest Procedure

Procedure

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest IS
BEGIN
    UPDATE Accounts
    SET Balance = Balance + (Balance * 0.01)
    WHERE AccountType = 'SAVINGS';
    COMMIT;
END;
/
```

Execute

```
BEGIN
    ProcessMonthlyInterest;
END;
/
```

Scenario 2: Update Employee Bonus

Procedure

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus
(
    p_DepartmentID IN NUMBER,
    p_BonusPercent IN NUMBER
)
IS
BEGIN
    UPDATE Employees
    SET Salary = Salary + (Salary * p_BonusPercent / 100)
    WHERE DepartmentID = p_DepartmentID;
```

```
    COMMIT;  
END;  
/
```

Execute

```
BEGIN  
    UpdateEmployeeBonus(10, 5);  
END;  
/
```

Scenario 3: Transfer Funds

Procedure

```
CREATE OR REPLACE PROCEDURE TransferFunds  
(  
    p_FromAccount IN NUMBER,  
    p_ToAccount IN NUMBER,  
    p_Amount IN NUMBER  
)  
IS  
    v_Balance NUMBER;  
BEGIN  
    SELECT Balance  
    INTO v_Balance  
    FROM Accounts  
    WHERE AccountID = p_FromAccount;  
    IF v_Balance >= p_Amount THEN  
        UPDATE Accounts  
        SET Balance = Balance - p_Amount  
        WHERE AccountID = p_FromAccount;  
        UPDATE Accounts  
        SET Balance = Balance + p_Amount  
        WHERE AccountID = p_ToAccount;  
        COMMIT;  
        DBMS_OUTPUT.PUT_LINE('Transfer Successful');  
    ELSE  
  
        DBMS_OUTPUT.PUT_LINE('Insufficient Balance');  
    END IF;  
END;  
/  
  
Execute  
  
SET SERVEROUTPUT ON;  
  
BEGIN  
    TransferFunds(101,102,5000);  
END;  
/
```

OUTPUT:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  1: dash

[      4 containers successful ]
[      4 tests found          ]
[      0 tests successful     ]
[      0 tests skipped        ]
BUILD SUCCESSFUL -----
PS C:\Users\angel\Desktop\dn\dn> exec sp_ProcessMonthlyInterest;
Procedure PROCESSMONTHLYINTEREST compiled.
PL/SQL procedure successfully completed.
2 rows updated.
PS C:\Users\angel\Desktop\dn\dn> exec sp_UpdateEmployeeBonus;
Procedure UPDATEEMPLOYEEBONUS compiled.
PL/SQL procedure successfully completed.
3 rows updated.
PS C:\Users\angel\Desktop\dn\dn> exec sp_TransferFunds;
Procedure TRANSFERFUNDS compiled.
Transfer Successful
PL/SQL procedure successfully completed.
PS C:\Users\angel\Desktop\dn\dn> █
```

TDD USING JUNIT5 AND MOCKITO

Exercise 1: Setting Up JUnit

Scenario:

You need to set up JUnit in your Java project to start writing unit tests.

Steps:

1. Create a new Java project in your IDE (e.g., IntelliJ IDEA, Eclipse).
2. Add JUnit dependency to your project. If you are using Maven, add the following to your pom.xml:

```
<dependency>
<groupId>junit</groupId>
<artifactId>junit</artifactId>
<version>4.13.2</version>
<scope>test</scope>
</dependency>
```

3. Create a new test class in your project.

Exercise 3: Assertions in JUnit

Scenario:

You need to use different assertions in JUnit to validate your test results.

Steps:

1. Write tests using various JUnit assertions.

Solution Code:

```
public class AssertionsTest {

    @Test

    public void testAssertions() {

        // Assert equals
        assertEquals(5, 2 + 3);

        // Assert true
        assertTrue(5 > 3);

        // Assert false
        assertFalse(5 < 3);

        // Assert null
        assertNull(null);
    }
}
```



```
// Assert not null
assertNotNull(new Object());
}
}
```

Exercise 4: Arrange-Act-Assert (AAA) Pattern, Test Fixtures, Setup and Teardown Methods in JUnit

Scenario:

You need to organize your tests using the Arrange-Act-Assert (AAA) pattern and use setup and teardown methods.

Steps:

1. Write tests using the AAA pattern.
2. Use `@Before` and `@After` annotations for setup and teardown methods.

Exercise 1: Mocking and Stubbing

Scenario:

You need to test a service that depends on an external API. Use Mockito to mock the external API and stub its methods.

Steps:

1. Create a mock object for the external API.
2. Stub the methods to return predefined values.
3. Write a test case that uses the mock object.

Solution Code:

```
import static org.mockito.Mockito.*;
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;

public class MyServiceTest {

    @Test

    public void testExternalApi() {
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);
        when(mockApi.getData()).thenReturn("Mock Data");

        MyService service = new MyService(mockApi);

        String result = service.fetchData();
```

```
assertEquals("Mock Data", result);  
}  
}
```

Exercise 2: Verifying Interactions

Scenario:

You need to ensure that a method is called with specific arguments.

Steps:

1. Create a mock object.
2. Call the method with specific arguments.
3. Verify the interaction.

Solution Code:

```
import static org.mockito.Mockito.*;  
import org.junit.jupiter.api.Test;  
import org.mockito.Mockito;  
public class MyServiceTest {  
    @Test  
    public void testVerifyInteraction() {  
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);  
        MyService service = new MyService(mockApi);  
        service.fetchData();  
        verify(mockApi).getData();  
    }  
}
```

SOLUTION:

Exercise 1: Setting Up JUnit 5

Maven Dependencies

```
<dependencies>  
    <!-- JUnit 5 -->  
    <dependency>  
        <groupId>org.junit.jupiter</groupId>  
        <artifactId>junit-jupiter</artifactId>  
        <version>5.10.2</version>  
        <scope>test</scope>
```

```
</dependency>
</dependencies>
```

Simple Test Class

```
import org.junit.jupiter.api.Test;
public class DemoTest {
    @Test
    void testMethod() {
        System.out.println("JUnit is working");
    }
}
```

Exercise 3: Assertions in JUnit 5

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
public class AssertionsTest {
    @Test
    void testAssertions() {
        assertEquals(5, 2 + 3);
        assertTrue(5 > 3);
        assertFalse(5 < 3);
        assertNull(null);
        assertNotNull(new Object());
    }
}
```

Exercise 4: AAA Pattern + Setup + Teardown

Calculator Class

```
public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }
}
```

Test Class

```
import org.junit.jupiter.api.*;

import static org.junit.jupiter.api.Assertions.*;
public class CalculatorTest {
    Calculator calculator;
    @BeforeEach
    void setup() {
        calculator = new Calculator();
        System.out.println("Setup");
    }
    @AfterEach
    void teardown() {
        System.out.println("Teardown");
    }
}
```

```

@Test
void testAdd() {
    // Arrange
    int a = 10;
    int b = 20;
    // Act
    int result = calculator.add(a, b);
    // Assert
    assertEquals(30, result);
}
}

```

Mockito Exercise 1: Mocking and Stubbing

ExternalApi

```

public interface ExternalApi {
    String getData();
}

```

MyService

```

public class MyService {
    private ExternalApi api;
    public MyService(ExternalApi api) {
        this.api = api;
    }

    public String fetchData() {
        return api.getData();
    }
}

```

Test

```

import org.junit.jupiter.api.Test;
import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.*;
public class MyServiceTest {
    @Test
    void testExternalApi() {
        ExternalApi mockApi = mock(ExternalApi.class);
        when(mockApi.getData())
            .thenReturn("Mock Data");
        MyService service = new MyService(mockApi);
        String result = service.fetchData();
        assertEquals("Mock Data", result);
    }
}

```

Mockito Exercise 2: Verifying Interactions

```

import org.junit.jupiter.api.Test;
import static org.mockito.Mockito.*;
public class MyServiceTest {

```

```

@Test
void testVerifyInteraction() {
    ExternalApi mockApi = mock(ExternalApi.class);
    MyService service = new MyService(mockApi);
    service.fetchData();
    verify(mockApi).getData();
}
}

```

OUTPUT:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  1: dash
PS C:\Users\angel\Desktop\dn\dn>
Testing started at 10:15 AM...

✓ AssertionsTest
  ✓ testAssertions()
  ✓ testAssertions()

-----
Test run finished after 42 ms
Test run finished after 42 ms
[      1 containers found      ]
[      1 containers successful ]
[      1 tests found          ]
[      1 tests successful      ]
[      1 tests successful      ]
[      1 tests successful      ]
[      0 tests failed          ]
-----

```

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  1: dash
PS C:\Users\angel\Desktop\dn\dn>
Setup
Teardown

✓ CalculatorTest
  ✓ testAdd()

-----
Test run finished after 35 ms

[      1 tests found          ]
[      1 tests successful      ]
[      1 tests successful      ]
[      0 tests failed          ]
-----

BUILD SUCCESSFUL
PS C:\Users\angel\Desktop\dn\dn> 

```

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  1: dash
PS C:\Users\angel\Desktop\dn\dn>
Setup
✓ AssertionsTest
  ✓ testAssertions()
✓ CalculatorTest
✓ MyServiceTest
  ✓ testExternalApi()
✓ MyServiceTest
  ✓ testVerifyInteraction()

-----
Test run finished after 125 ms
[      4 containers found      ]
[      4 containers successful ]
[      4 tests found          ]
[      4 tests successful      ]
[      0 tests failed          ]
[      0 tests skipped         ]
-----

BUILD SUCCESSFUL

```

SLF4J LOGGING FRAMEWORK

Exercise 1: Logging Error Messages and Warning Levels

Task: Write a Java application that demonstrates logging error messages and warning levels using SLF4J.

Step-by-Step Solution:

1. Add SLF4J and Logback dependencies to your `pom.xml` file:

```
<dependency>
<groupId>org.slf4j</groupId>
<artifactId>slf4j-api</artifactId>
<version>1.7.30</version>
</dependency>
<dependency>
<groupId>ch.qos.logback</groupId>
<artifactId>logback-classic</artifactId>
<version>1.2.3</version>
</dependency>
```

2. Create a Java class that uses SLF4J for logging:

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
public class LoggingExample {
    private static final Logger logger = LoggerFactory.getLogger(LoggingExample.class);
    public static void main(String[] args) {
        logger.error("This is an error message");
        logger.warn("This is a warning message");
    }
}
```

PROGRAM:

pom.xml Dependencies

```
<dependencies>
    <dependency>
        <groupId>org.slf4j</groupId>
        <artifactId>slf4j-api</artifactId>
        <version>1.7.30</version>
    </dependency>
    <dependency>
        <groupId>ch.qos.logback</groupId>
```

```

        <artifactId>logback-classic</artifactId>
        <version>1.2.3</version>
    </dependency>
</dependencies>

```

LoggingExample.java

```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
public class LoggingExample {
    private static final Logger logger =
        LoggerFactory.getLogger(LoggingExample.class);
    public static void main(String[] args) {
        logger.error("This is an error message");
        logger.warn("This is a warning message");
    }
}

```

OUTPUT:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL
[ 4 containers successful ]
[ 4 tests found ]
[ 0 tests successful ]
[ 0 tests skipped ]
BUILD SUCCESSFUL -----
PS C:\Users\angel\Desktop\dn\dn> exec sp_ProcessMonthlyInterest;
Procedure PROCESSMONTHLYINTEREST compiled.
PL/SQL procedure successfully completed.
2 rows updated.
PS C:\Users\angel\Desktop\dn\dn> exec sp_UpdateEmployeeBonus;
Procedure UPDATEEMPLOYEEBONUS compiled.
PL/SQL procedure successfully completed.
3 rows updated.
PS C:\Users\angel\Desktop\dn\dn> exec sp_TransferFunds;
Procedure TRANSFERFUNDS compiled.
PS C:\Users\angel\Desktop\dn\dn> ERROR LoggingExample - This is an error message
PS C:\Users\angel\Desktop\dn\dn> WARN LoggingExample - This is a warning message
PS C:\Users\angel\Desktop\dn\dn>

```